Competitive Programming Techniques

Invariants, operation mappings, and counting shortcuts for strings, arrays, ranges, and integers.

How to use. Start with the allowed move. Ask what survives, what becomes simpler in different coordinates, and what proves a target reachable. Each entry states the useful reduction and the assumptions needed for it. Dynamic programming is outside this outline.

Notation. Arrays and strings use indices $1, ..., n$ with $n \geq 1$ unless stated otherwise. Ranges are inclusive, $P_0 = 0$, and $[Q]$ is 1 if statement Q is true and 0 otherwise. A subarray/substring is contiguous; a subsequence need not be. Costs count individual allowed operations. Math uses unbounded integers; implementations need wide enough types.

Original reduced-string notes. Here "reduced binary string" means **run compression**: replace each maximal constant run by one bit. Its behavior differs from deleting equal or opposite pairs; all three rules are treated separately below.

Contents

1 Finding the observation 1
2 Array operations and invariants 2
3 Strings 4
4 Binary strings 5
5 Prefix transforms and range sums 7
6 Number theory 9
7 Bitwise observations 11
8 Worked observation chains 13
9 Final checks before coding 13
10 References for deeper proofs 13

1 Finding the observation

1.1 An operation-analysis checklist

- **Write the change, not the whole state.** For a move $a \to a'$, calculate $a' - a$ (or bitwise XOR). If only a few coordinates change, expand candidate quantities using only those coordinates. Example: $(x, y) \to (x + d, y - d)$ changes the sum by zero.
- **Try cheap invariants first.** Length, multiset, sum, sum modulo k, XOR, parity of each value, counts of each character/bit, product sign, gcd, relative order, and the first/last element. An operation may preserve one of these only inside each connected component or index class.
- **Try weighted invariants.** A linear quantity $\sum_i w_i a_i$ survives a move vector v exactly when $\sum_i w_i v_i = 0$. The same test modulo m gives modular invariants. For adjacent simultaneous increments, alternating weights work; for transfers, equal weights work.
- **Solve for how often moves occur.** If moves add fixed vectors, write target minus start as Mx. With reversible moves, x may be any integer vector; with forward-only moves, require $x \geq 0$. Bit flips use arithmetic modulo 2. Bounds on intermediate states are an additional constraint.
- **Necessary is not sufficient.** A preserved sum rules out targets with different sums. It does not prove equal-sum targets are reachable. Prove the converse with a construction, a canonical form, or an exact characterization of move counts.
- **Invariant versus potential.** An invariant is constant; a potential moves monotonically. Inversions, number of runs, total absolute deviation, and number of nonzero entries often bound progress. Strictly decreasing **nonnegative integer** potential proves termination; a decreasing real number need not.
- **Quotient out irrelevant detail.** If only relative order matters, compress ranks; if only parity matters, take residues; if only transitions matter, compress runs; if only differences matter, subtract a reference. Prove the transformed state retains everything needed for the query.
- **Look at the support of a move.** A point edit affects adjacent boundaries; a range addition affects two differences; an adjacent transfer affects one prefix sum. Choose coordinates in which each move becomes sparse.
- **Do moves commute?** Fixed additions and fixed-mask XORs commute; sum or XOR their effects. Assignments and swaps that share an index generally do not. "Only the number of operations matters" needs a commutativity argument.
- **Normalize a target.** Equalize to zero by subtracting a common offset; turn a target bitstring into zero by XORing with it; turn sorting into a permutation of target positions. Always check that the move rules survive this change of coordinates.
- **Work backwards.** Deletion becomes insertion, division becomes multiplication, and "set to the smaller value" becomes a choice of predecessors. Backward construction helps when forward branching is large but the final state has a forced last move.
- **Use a witness.** Fix a minimum, maximum, first mismatch, last mismatch, or rarest value. Many pairwise conditions then reduce to comparisons against this one element. Example: gcd of all pairwise differences equals gcd of differences from one reference.
- **Prove a greedy move locally.** Compare two adjacent choices and exchange their order; show one order cannot hurt. Or choose a move that settles one position permanently. A successful sample is not an exchange argument.
- **Brute-force the conjecture.** Enumerate tiny states and reachable states to distinguish "necessary" from "iff"; include duplicates, zeros, negative values, endpoints, and length one. Search for the smallest counterexample before optimizing.

1.2 Fast operation-to-model map

Operation / signal	Useful model
Arbitrary legal swaps	Position graph; component multisets
Adjacent swaps	Inversions; ordered transport
Add on an interval	Two difference-array impulses
Transfer across neighbors	Prefix imbalance / flow
Flip a binary interval	Toggle its two boundaries
Merge adjacent blocks	A partition into original intervals
Repeat a fixed permutation	Cycles; exponent modulo cycle length
Add equal values to neighbors	Alternating weighted sum
Delete opposite bits	Signed count / balance
Apply the same XOR to two entries	Total XOR; one equation per bit
Global translation	Differences from an anchor
Huge integer values	Residues, divisors, bit length, or ranks

2 Array operations and invariants

2.1 Rearrangement and order

- **Arbitrary swaps preserve exactly the multiset.** Sorting both arrays gives a canonical test for reachability when every pair may be swapped. Equal sum, XOR, or gcd is weaker: $[1, 4]$ and $[2, 3]$ have equal sums but cannot be connected by swaps.
- **Allowed swaps form a graph.** Put an edge between positions that may be swapped, assuming legality is fixed independently of values. Within a connected component, edge swaps generate every permutation. Thus the multiset in each component is a complete reachability test.
- **Swaps at fixed distance.** On a line, swapping positions i and $i + k$ splits positions by residue modulo k. On a cycle of length n, edges of offset k give $\gcd(n, k)$ components. Compare source and target multisets in these classes.
- **Restricted parity swaps.** If only equal-parity positions can swap, preserve separate multisets at odd/even positions. If only **values** of a certain kind can swap, the position graph may change; do not apply a static-component argument without checking the rule.
- **Adjacent swaps count inversions.** For a permutation, the minimum adjacent swaps to sort is its inversion count. Each adjacent out-of-order swap removes exactly one. With duplicate values, pair equal occurrences in stable order before counting inversions to a target.
- **Arbitrary swaps count cycles.** For a permutation of distinct labels, minimum unrestricted swaps to reach identity is $n - c$, where c includes one-element cycles. A swap can increase the cycle count by at most one, and a cycle of length k takes $k - 1$ swaps.
- **Duplicates complicate cycle minima.** When equal values are interchangeable, different matchings to sorted positions can produce different cycle counts. Any fixed matching gives a construction; it need not give the fewest arbitrary swaps.
- **Permutation parity.** Every transposition toggles parity. If exactly k swaps are required and a solution uses d swaps, extra pairs of identical swaps pad the length by 2 when some swap is available. Duplicates may hide label parity, and a state with no legal move cannot be padded.
- **Reversal parity.** Reversing a block of length k is $\left\lfloor \frac{k}{2} \right\rfloor$ transpositions, so its permutation sign is determined by $\left\lfloor \frac{k}{2} \right\rfloor$ modulo 2. A three-element reversal swaps its endpoints, hence preserves the multiset on each index parity.
- **Rotation order.** Rotating length $n > 0$ by k positions returns every position after $\frac{n}{\gcd(n,k)}$ applications. A string with repeated values can return sooner; the position permutation and the observed array need not have the same period.
- **Repeated permutations.** Split a fixed permutation into cycles, then shift each cycle by the repetition count modulo its length. This gives $O(n)$ work after cycle discovery even for enormous counts. Permutation powers.
- **Subsequence order is rigid.** Deleting entries preserves the relative order of survivors. Swapping only adjacent unequal types may preserve the internal order within each type; label occurrences to see what cannot cross.
- **Sorting may discard the question.** Sort when feasibility/cost depends only on the multiset or when you can restore indices. Sorting destroys adjacency, contiguous ranges, and original subsequence order.

2.2 Additions, transfers, and equalization

- **Independent changes by a fixed step.** With moves $a_i \to a_i + -d$ for $d > 0$, each coordinate residue modulo d is invariant. A target is reachable iff the corresponding residues agree when intermediate integer values are unrestricted. Minimum moves is $\sum_i \frac{|a_i - b_i|}{d}$.
- **Equalize by fixed-step changes.** All values must share one residue modulo d. After scaling into that residue class, minimize total unit moves with a median. With only increases, the cheapest target is the maximum; with only decreases, it is the minimum.
- **Independent unit edits.** To reach a fixed integer target array, cost is $\sum_i |a_i - b_i|$. If each move may edit an arbitrary subset of coordinates by one in either chosen direction per coordinate, cost becomes $\max_i |a_i - b_i|$; changing the allowed subset changes the metric.
- **Transfer one unit between any two positions.** Sum is invariant and is sufficient for unrestricted integer states. For equal-sum source/target, minimum cost is $\sum_i \max(a_i - b_i, 0) = \frac{1}{2} \sum_i |a_i - b_i|$. Each move settles one unit of surplus and deficit simultaneously.
- **Transfers on a graph.** Component sums are invariant. With connected edges and reversible unit transfers, any integer target of the same component sum is reachable by routing surplus. Nonnegative source and target can also be handled without going negative if there are no capacities.
- **One-way neighbor transfers.** Let $D_i = \sum_{j=1}^{i} (a_j - b_j)$. If only moves $i \to i + 1$ are allowed, reachability requires total difference zero and $D_i \geq 0$ for every proper prefix. These are also sufficient without capacities; D_i is the forced flow across edge i.
- **Add to every entry.** Every pairwise difference stays fixed. Two arrays are connected by global additions iff $b_i - a_i$ is the same allowed shift at every position. Normalize by subtracting the first entry or minimum.
- **Increment all except one.** Modulo a global shift, this is decrementing the excluded entry. Minimum moves to make all entries equal is $\sum_i a_i - n \min_i a_i$. Use the original minimum as the normalized target; the actual final value includes the accumulated global shift.
- **Increment one entry and decrement another by the same amount.** Sum stays fixed even when the amount varies. If the amount must be a multiple of d, coordinate residues modulo d are additional constraints.
- **Add one to both endpoints of a path edge.** Alternating sum $a_1 - a_2 + a_3 - \ldots$ stays fixed. For target difference $d_i = b_i - a_i$, write $x_0 = x_n = 0$ and $d_i = x_{i-1} + x_i$. The forced recurrence $x_i = d_i - x_{i-1}$ determines all move counts. With signed moves, the final boundary equation is sufficient; with additions only, also require all $x_i \geq 0$.
- **Equal increments on graph edges.** On a bipartite graph, the sum on one color minus the sum on the other is invariant. On every graph, total-sum parity is invariant. An odd cycle destroys the alternating-color invariant; do not assume a path argument works on every graph.
- **Add a fixed vector repeatedly.** After k steps, the state is $a + kv$. All nonzero coordinates of v must imply the same integer k, and zero coordinates must already match. One-way repetition requires $k \geq 0$.
- **Global affine operations.** A sequence of maps $x \to ux + v$ is again $x \to Ax + B$. Applying (u, v) after (A, B) gives $(uA, uB + v)$. Maintain two parameters; composition order matters, and modular inversion needs invertible multipliers.
- **Anchored differences.** Common translations disappear in $a_i - a_1$. Common multiplication scales all differences. Ratios of nonzero differences may survive an affine transformation, but exact integer arithmetic and zero cases matter.
- **All-pairs equalization is not always averaging.** A fixed sum forces a common integer value to be $\frac{S}{n}$, so n must

divide S. Divisibility alone does not settle restrictions such as fixed-step changes, one-way flow, or forbidden positions.

2.3 Local rules and information loss

- **Merge adjacent values by summing.** Length falls but total sum survives. Every final entry is the sum of a nonempty contiguous block of the original array, and every such partition is realizable. The operation is a choice of cuts.
- **Positive equal-sum blocks.** With strictly positive entries and desired block sum T, scan and cut when the running sum reaches T; exceeding T proves impossibility. With negative values this greedy argument fails. If all blocks have equal integer sum, their count divides the total except for the special zero-total case.
- **Split into two positive summands.** Total sum survives and length increases by one. A positive integer x can become any ordered composition of x. Thus the final sequence must refine each original entry into consecutive pieces, not arbitrarily reorder mass.
- **Euclidean replacement preserves gcd.** Replacing one coordinate x by $x - qy$ while retaining y preserves the gcd of the whole array. Replacing a pair by their single difference does **not**: $[6, 10] \to [4]$ changes gcd from 2 to 4.
- **Min/max propagation.** If a chosen entry may be replaced by the minimum of itself and a neighbor, no new value appears, values never increase, and a global minimum can spread across a connected graph. Analogous statements hold for maximum propagation.
- **Averaging a pair.** Replacing real x, y by their common mean preserves sum, keeps every value within the old global range, and decreases $\sum_i a_i^2$ by $\frac{(x-y)^2}{2}$. Integer rounding may destroy the sum invariant unless explicitly compensated.
- **Replace a pair by AND and OR.** Per-bit numbers of set bits survive, so the numeric sum survives. Write $x = c + p$, $y = c + q$, where c is the common-bit part. The new pair is $(c, c + p + q)$ and its sum of squares increases by $2pq \geq 0$.
- **Concentrate bits for convex objectives.** Under unrestricted pairwise AND/OR replacement on nonnegative fixed-width integers, bits can be packed into nested sets: the jth value receives every bit whose frequency is at least j. A sorting network applied bitwise realizes this arrangement; it maximizes the sum of squares by aligning large contributions.
- **Same XOR on two entries.** Applying an arbitrary mask to two entries preserves their combined XOR, hence total XOR. On a connected graph with arbitrary-mask edge moves, equality of total XOR is sufficient: settle leaves toward a root. With a **fixed** mask there are more constraints.
- **Flip signs of any pair.** Absolute values survive. With no zeros, parity of the number of negative entries survives and is sufficient for reaching sign patterns using arbitrary pair flips. On a connected edge graph, any even set of sign toggles is also reachable.
- **Maximum sum under pair sign flips.** For at least two positions and unrestricted pair flips, use $\sum_i |a_i|$. If the negative count is odd and no zero exists, subtract $2 \min_i |a_i|$. A zero absorbs an otherwise unwanted sign toggle. The same formula handles length one directly.
- **Modulo can halve a value.** For positive integers x, m, a changing operation $x \to x \bmod m$ gives a result less than $\frac{x}{2}$: if $m \leq \frac{x}{2}$ it is below m; otherwise it equals $x - m$. Hence each positive entry changes only $O(\log(x + 1))$ times under repeated modulo updates unless other operations increase it.
- **Floor division shrinks quickly.** Repeated $x \to \left\lfloor \frac{x}{k} \right\rfloor$ for fixed integer $k \geq 2$ reaches zero in $O(\log(x + 1))$ steps for nonnegative x. Group values by the quotient or process only times at which the value changes.
- **Saturation makes most work irrelevant.** Repeated $x \to \min(x, c)$ is idempotent. A series of such caps collapses to the smallest cap; repeated lower bounds collapse to the largest. Mixed caps, floors, and assignments require respecting order.

2.4 Sorting, extrema, and greedy structure

- **Coordinate compression.** Replace each distinct value by its rank when only equality and order matter. This preserves comparisons and inversion counts, but not sums, distances, residues, or gcd. Keep original values for arithmetic weights.
- **Nearest pair is adjacent after sorting.** The smallest absolute difference among numeric entries occurs between sorted neighbors. The largest is maximum minus minimum. This does not preserve distances between **original adjacent positions**.
- **Median minimizes absolute deviation.** Any median minimizes $\sum_i |a_i - x|$ over real x; for even n, the interval between the middle two works. With nonnegative weights and positive total weight, weighted absolute deviation uses a weighted median. Greedy sorting notes.
- **Mean minimizes squared deviation.** Over reals, $\sum_i (a_i - x)^2$ is minimized at the mean. For an integer target, test floor/ceiling of the mean. Confusing the mean with the median changes the objective.
- **Midrange minimizes the worst deviation.** The best real center for $\max_i |a_i - x|$ is $\frac{\min a + \max a}{2}$; test its floor/ceiling for integers. Only the two extreme values matter.
- **Sorted matching minimizes total absolute distance.** Given equal-size multisets a, b, pair their sorted orders to minimize $\sum_i \left| a_i - b_{\pi(i)} \right|$. Crossing pairs can be uncrossed without increasing cost. For labelled or restricted pairings, this freedom may be absent.
- **Rearrangement inequality.** For two sorted real sequences, matching the same order maximizes their dot product; opposite order minimizes it. Swapping a crossed pair changes the objective by a product of two differences, giving the exchange proof.
- **Minimize the largest pair sum.** To split an even-size sorted array into pairs minimizing the maximum pair sum, pair the smallest with the largest. Exchange partners of the two extremes to prove an optimal pairing can take this form.
- **Minimum range for a chosen size.** To choose k values minimizing maximum minus minimum, sort and scan consecutive windows of length k. Any chosen set's extremes span at least one such window.
- **Thresholding converts values to counts.** "At least k elements are at least x" is a frequency/rank question. "Raise all entries below x to x" costs $\sum_i \max(0, x - a_i)$, computable from a sorted prefix sum and the number below x.
- **Water filling has breakpoints.** When increments are shared among the smallest values, the active group grows only at the next distinct sorted value. Raise a whole group to the next level, or spend the remaining budget evenly within it.
- **Absolute values become linear in regions.** Once the order of x relative to all a_i is known, $\sum_i |a_i - x|$ is linear in x. Candidate optima often occur at breakpoints; adding a quadratic term changes this to piecewise quadratic.
- **A local edit changes local adjacency terms.** If cost is $\sum_{i=1}^{n-1} f(a_i, a_{i+1})$, changing a_i affects at most two terms; swapping two positions affects at most four distinct edges. Deduplicate edges for adjacent positions and endpoints.
- **Reversing an interval preserves its internal undirected edges.** For a symmetric edge cost $f(x, y) = f(y, x)$, an interval reversal changes only the edges crossing the interval's boundaries. Directed costs can change all internal terms.

- **Deletion reconnects neighbors.** Removing an internal element replaces $f(x,y)+f(y,z)$ by $f(x,z)$. For absolute differences, triangle inequality says total variation cannot increase. Endpoint deletion removes only one term.
- **Adding a constant changes weighted sums predictably.** A range addition by d changes $\sum_i w_i a_i$ by d times the total weight in that range. Precompute prefix weights even if the array itself is dynamic.

2.5 Frequencies, mex, and pigeonholes

- **Pair counts live in frequencies.** Equal unordered pairs contribute $\sum_x \frac{c_{x(c_x-1)}}{2}$; unequal pairs contribute $\frac{n(n-1)}{2}$ minus this. Inserting one more copy of x adds exactly c_x equal pairs.
- **Cancel distinct values in pairs.** Repeatedly removing two **different** values can remove everything iff n is even and the largest frequency is at most $\frac{n}{2}$. More generally the minimum number left is $\max(n \bmod 2, 2c_{\max} - n)$ when pairs may be chosen anywhere.
- **A strict majority survives unlike-pair cancellation.** A value occurring more than $\frac{n}{2}$ times cannot be eliminated completely by deleting pairs of different values. A resulting candidate still needs a frequency check when existence of a majority is not promised.
- **Mex is small.** The smallest absent nonnegative integer in an array of length n lies in $[0,n]$. Negative values and values above n do not directly affect it, though they can supply replaceable entries.
- **Make mex exactly k by replacement.** For $0 \le k \le n$, let m be the number of missing values in $[0,k-1]$ and c the count of k. With arbitrary single-entry replacement, minimum moves is $\max(m,c)$: use copies of k to fill missing smaller values first.
- **Mex at least k.** This means every value $0,1,...,k-1$ occurs. It is a conjunction of presence constraints; it does not require a particular frequency or sorted order. For a partition into q nonempty groups each with mex at least k, each of these values must occur at least q times; with arbitrary grouping and $1 \le q \le n$, that condition is sufficient. Contiguous groups need additional order information.
- **Missing values can force a witness.** If $n+1$ objects enter n residue classes, a collision exists. For prefix sums modulo n, this gives a nonempty divisible-sum subarray; for positions on a circle, residues often reveal a shorter representative.
- **Small alphabet suggests counts, not positions.** If a condition depends only on multiplicities, the state space is a frequency vector of alphabet size. If position matters, keep counts alongside the minimal order information (runs, first/last occurrences, or prefix counts).
- **Large versus small frequency split.** There are at most $\frac{n}{B}$ values with frequency greater than B. Treat these few heavy values individually and process light values by their occurrences; first derive the two costs, then choose B to balance them.
- **Choose a rare witness.** If a valid answer must contain every required symbol, start from occurrences of the rarest required symbol. This bounds candidate anchors, but it is only a speedup if the rest can be checked efficiently.
- **Complement counting.** Count all objects and subtract invalid ones when invalidity has a simpler witness: equal pairs, all-zero subarrays, missing letters, or intervals avoiding a marked position. Overlapping bad events require inclusion-exclusion or a unique owner.

3 Strings

Positions in this section are $1,...,n$. A substring is contiguous; a subsequence preserves order but may skip positions. State whether an operation changes positions, characters, or both.

3.1 Counts, order, and palindromes

- **Arbitrary swaps → histogram.** Swaps preserve every character count, and equal histograms are sufficient for reachability when any pair may be swapped. Sorting gives a canonical representative of each reachable class.
- **Restricted swaps → components.** For a fixed set of legal position pairs, put an edge between positions that may be swapped. The multiset inside every connected component is invariant and sufficient: edge swaps generate every permutation within a component. Swapping positions distance k apart on a line gives residue classes modulo k.
- **Adjacent swaps → stable matching.** Match the first occurrence of each character in the source to its first occurrence in the target, and so on. For anagrams, the minimum adjacent swaps equals the inversion count of this destination-position sequence. Crossing identical characters never helps.
- **Replacement versus rearrangement.** With positions fixed, minimum single-character replacements is Hamming distance. If free rearrangement is also allowed, it becomes half the sum of absolute histogram differences, for equal-length strings. Deletions from both strings to obtain anagrams cost the full sum of histogram differences.
- **Palindrome permutation.** A multiset can form a palindrome iff at most one frequency is odd. With deletions and free rearrangement, the minimum deletions is $\max(0, o-1)$, where o is the number of odd frequencies. This does not solve palindrome subsequences.
- **Fixed-order palindrome repair.** Count unequal mirrored pairs. Exactly one replacement per unequal pair is necessary and sufficient. For a palindrome obtainable by changing at most one character, this count must be at most one.
- **Count palindrome permutations.** When feasible, let $h_c = \left\lfloor \frac{f_c}{2} \right\rfloor$. The number of distinct palindromic arrangements is $\frac{\left(\lfloor \frac{n}{2} \rfloor\right)!}{\prod_c h_c!}$: arrange the left half; the middle and right half are forced.
- **Avoid equal neighbors.** A multiset admits an arrangement with no equal adjacent characters iff its largest frequency is at most $\lceil \frac{n}{2} \rceil$. The most frequent character needs enough separators. This criterion is for a line; a circle needs separate endpoint handling.
- **Assemble repeated target words.** If letters may be rearranged freely, the maximum number of copies of target t is $\min_{c: f_{t(c)}>0} \left\lfloor \frac{f_{s(c)}}{f_{t(c)}} \right\rfloor$. Without rearrangement, counts alone do not guarantee subsequences or substrings.

3.2 Deletion, matching, and local structure

- **Subsequence feasibility.** Greedily match each target character at its earliest available source position. Earlier matches leave at least as much suffix for all remaining characters. For many queries, store occurrence lists and binary-search the next position, or precompute next-occurrence indices.
- **Delete one contiguous block.** If s is longer than target t, let p be their longest common prefix length and q their longest common suffix length. Deleting one block from s yields t iff $p+q \ge |t|$. Choose a split of t covered by the prefix and suffix.
- **Lexicographically smallest after k deletions.** Maintain a stack; while deletions remain and its last character exceeds the next character, pop. Then append the next character.

Spend any leftover deletions at the end. The first improved position dominates every later choice.

- **Run compression.** Replace each maximal equal-character run by one character, e.g. `aaabbcca` becomes `abca`. This preserves the sequence of changes, but loses run lengths and counts. It is useful only when the operation or objective can be expressed using that information.
- **Fixed-pattern updates are local.** Changing position i can affect occurrences of a pattern of length m only at starts $i-m+1, ..., i$. Subtract old contributions, mutate, then add new contributions. For constant m, each update takes constant time.
- **Palindrome extension at one end.** The shortest append-only extension to a palindrome preserves the longest palindromic suffix and appends the reverse of the preceding prefix. Find that suffix using the prefix function on `reverse(s) + separator + s`; the separator must be absent from the alphabet.

3.3 Transformations and equivalence

- **Consistent renaming → equality pattern.** Under a bijective renaming of characters, only which positions are equal matters. Canonicalize characters by first appearance: `paper` and `title` both become `01023`. Check both mapping directions; a one-way map accidentally permits merging characters.
- **Global alphabet shift → differences.** Encode letters modulo alphabet size q. Two equal-length strings differ by one uniform cyclic shift iff every $t_i - s_i$ is the same modulo q. Equivalently, their adjacent modular differences agree; the initial character fixes the shift.
- **Uniform character map commutes with positions.** A fixed character substitution or binary complement commutes with reversal, rotation, and any fixed position permutation. Move these operations together and combine them; this fails for substitutions dependent on current positions or neighbors.
- **Repeated transformations → cycles.** A fixed permutation of positions decomposes into cycles. Applying it k times shifts each cycle by k modulo its length. Reverse twice is identity; rotating repeatedly reduces the total shift modulo n.
- **Rotation → doubled string.** Equal-length t is a cyclic rotation of s iff it occurs in `s+s`. Search only starting positions below n when counting rotations. The number of distinct rotations can be less than n because of periodicity.
- **Fixed-step rotations.** If the only allowed operation rotates by k, reachable offsets are exactly multiples of $\gcd(n,k)$ modulo n. There are $\frac{n}{\gcd(n,k)}$ reachable offsets; repeated contents can make several offsets produce the same string.
- **Reversal identities.** Reverse each of blocks A, B, then reverse their concatenation: the result is BA. This implements a rotation using three reversals. If only whole-string reversal is allowed, a canonical representative is the smaller of s and its reverse.
- **Cancellation is not run compression.** Repeatedly deleting adjacent equal pairs is computed by a stack: pop an equal top, otherwise push. The final reduced word is unique for this rule. Character-count parity is necessary but generally insufficient to determine it; `abab` has all even counts but does not reduce.
- **Do not generalize one rewrite rule.** Deleting `ab` only, deleting both `ab` and `ba`, and deleting `aa`/`bb` define different problems. On the alphabet consisting only of `a` and `b`, for `ab`-only deletion match each `b` to a preceding unmatched `a`; surviving letters have form `bbb...aaa...`. Other characters can block matches.

3.4 Borders, periods, and comparison

- **Border ↔ period.** A proper border of length b is an equal prefix and suffix. It gives period $p = n - b$: $s_i = s_{i+p}$ whenever both exist. A period need not divide n; `ababa` has period two but is not complete copies of `ab`.
- **Primitive repeating block.** Let b be the longest proper border and $p = n - b$. If p divides n, the primitive block has length p; otherwise the only whole-copy representation has block length n. In the divisible case there are p distinct cyclic rotations. Prefix-function reference.
- **All borders form a chain.** With π_i the longest border length of prefix $1, ..., i$, all proper borders of s are π_n, π_{π_n}, and so on until zero. Following these links skips lengths that cannot be borders.
- **Two periods force a smaller one.** If p, q are periods and $n \geq p + q - \gcd(p,q)$, then $\gcd(p,q)$ is also a period (Fine–Wilf). The length threshold is essential: short overlaps can support incompatible apparent repeating blocks. Proof, Lemma 1.
- **Commuting strings share a root.** For nonempty strings, `s+t == t+s` iff both are repetitions of a common primitive string. Thus a common whole-copy string divisor exists exactly under that equality; its longest possible length is $\gcd(|s|, |t|)$.
- **Maximal overlap → shortest ordered merge.** To append t after s while overlapping their boundary, find the longest suffix of s equal to a prefix of t. A prefix-function or Z computation on `t + separator + s` finds it. Internal containment must also be checked for general shortest-superstring tasks.
- **Lexicographic order is decided early.** Compare at the first differing character; if one string is a prefix, the shorter comes first. For optimal concatenation of nonempty strings, sort by `a+b < b+a`; sorting by `a < b` can fail, e.g. `b` and `ba`.
- **Minimal rotation as a canonical form.** To group strings up to rotation, use their lexicographically smallest rotation. Duval's algorithm obtains it in linear time using the doubled string; comparing every rotation naively can take quadratic time. Duval reference.
- **Fast equality is not exact equality.** Rolling hashes make substring comparisons cheap after preprocessing; align exponents or normalize lengths. Equal hashes can collide. Multiple independently chosen hashes reduce risk; prefix/Z algorithms provide deterministic alternatives for their specific matching tasks. Hashing reference.

4 Binary strings

Write bits as $s_i \in \{0, 1\}$; XOR is xor. Unless stated otherwise, operations have unit cost and any specified interval is allowed. Empty strings and circular strings need their own endpoint conventions.

4.1 Runs, boundaries, and counting

- **Only two compressed strings per positive length.** Binary run compression always alternates, so its length r and first bit determine it. There are exactly two compressed strings of each positive length. This says nothing about the 2^n uncompressed strings of length n.
- **Expanding a run preserves compression.** Replacing one bit by any positive-length run of that same bit leaves the compressed string unchanged. Inserting a constant run anywhere increases run count by at most two, since only the insertion boundaries can change.
- **One flip changes at most two boundaries.** Flipping an interior bit toggles both adjacent equality tests: run count

changes by $-2, 0$, or 2. Flipping an endpoint changes it by one in magnitude for $n > 1$; for $n = 1$, run count stays one.

- **Transitions telescope.** Let N_{01}, N_{10} count adjacent `01` and `10`. Then $N_{01} - N_{10} = s_n - s_1$. Hence transition directions alternate, their counts differ by at most one, and run count is $1 + N_{01} + N_{10}$ for a nonempty string.
- **Constant substrings split by runs.** A run of length L contributes $\frac{L(L+1)}{2}$ nonempty constant substrings. Sum over runs. For constant subsequences, use character counts instead; contiguity is what makes this run formula work.
- **Alternating subsequence → one per run.** The longest alternating subsequence has length equal to the number of runs: choose one bit from each. Therefore minimum deletions to make the string alternating is $n - r$. An alternating substring, however, depends on consecutive differing neighbors.
- **Basic construction counts.** There are 2^n binary strings of length n, and $\binom{n}{k}$ with exactly k ones. With z zeros and o ones, the count is $\binom{z+o}{o}$. Treat invalid binomial parameters as zero.
- **Choose run boundaries.** For $1 \le r \le n$, exactly $2\binom{n-1}{r-1}$ binary strings have r runs: choose the first bit and the $r-1$ boundaries. This is an easy way to count by transitions rather than by characters.
- **Fix counts and runs → compositions.** For a fixed starting bit and r runs, let r_0, r_1 be the numbers of zero/one runs. If both are positive, count $\binom{z-1}{r_0-1}\binom{o-1}{r_1-1}$ by distributing each bit count into positive run lengths. Handle an absent bit separately.
- **No adjacent ones → gaps.** With z zeros, place each of o ones into a different one of $z+1$ gaps. Feasibility is $o \le z+1$, and the number of strings is $\binom{z+1}{o}$. Stars-and-bars similarly handles minimum spacing by reserving required zeros first.
- **Assigning seats → binary construction.** Encode occupied seats by one and empty seats by zero. For a row with no neighboring occupied seats, forbid `11`; fitting k people into n seats is possible iff $k \le \lceil \frac{n}{2} \rceil$. With fixed occupants, first reject adjacent occupants, then forbid their neighbors and fill the remaining independent empty segments using gaps.
- **Ordered unequal pairs.** The numbers of `01` and `10` subsequences sum to zo: every zero-one pair contributes exactly one orientation. Complement or reversal swaps the two counts. Do not confuse these length-two subsequences with adjacent transitions.
- **Circular boundary parity.** A binary circle has an even number of transitions, because every change away from a bit needs a change back. A nonempty perfectly alternating circle exists iff its length is even. Linear formulas must include the last-to-first edge when moved to a circle.

4.2 Flips as XOR and differences

- **Reduce source-to-target to clearing mismatches.** Define $m_i = s_i \operatorname{xor} t_i$. Any operation consisting solely of flipping a specified set of positions acts on m in the same way. All flip operations commute, and using an identical operation twice cancels; only operation parity matters.
- **Complement preserves boundaries.** Complementing every bit preserves equalities, run lengths, palindrome status, and Hamming distances between two simultaneously complemented strings. It swaps zero/one counts. Many cases therefore need checking only once, then complementing the construction.
- **Interval flip → two boundary toggles.** Extend m with $m_0 = m_{n+1} = 0$ and define $d_i = m_i \operatorname{xor} m_{i-1}$ for $1 \le i \le n+1$. Flipping $[l, r]$ toggles only d_l, d_{r+1}. The number of set boundary bits is always even.
- **Minimum arbitrary interval flips.** To clear m, the answer is its number of one-runs, equivalently half the number of set bits in the extended boundary array. Each operation removes at most two boundaries; flipping each one-run achieves the bound.
- **Minimum prefix flips.** Process from right to left; once position i is fixed, shorter prefixes cannot disturb it. The optimum is $m_n + \sum_{i=1}^{n-1} [m_i \ne m_{i+1}]$. Equivalently, count boundaries against a trailing zero.
- **Minimum suffix flips.** Symmetrically, process left to right. The answer is $m_1 + \sum_{i=1}^{n-1} [m_i \ne m_{i+1}]$. A running XOR flag tracks whether the current bit has already been flipped an odd number of times.
- **Fixed-length window flips are forced.** For allowed length k, scan left to right while tracking active flip parity with an expiration difference array. If the current bit is wrong, the window starting there is forced. If it overhangs the end, transformation is impossible. This gives a linear-time minimum when each legal window may be used freely.
- **Flip adjacent pairs → total parity.** Flipping positions $i, i+1$ preserves XOR of all bits. On a path this condition is sufficient. If x_i records whether edge i is used, then $x_i = m_1 \operatorname{xor} ... \operatorname{xor} m_i$ for $i < n$; the final position checks feasibility. The minimum is the number of set x_i.
- **Flip any two distinct positions.** Reachability again requires an even mismatch count, which is sufficient by pairing mismatches. Minimum operations is half that count. Restricted permitted pairs instead preserve mismatch parity in every connected component of the permitted-edge graph; those conditions are sufficient within each component.

4.3 Swaps, transport, and alternating targets

- **Arbitrary swaps to a binary target.** Equal one-counts are necessary and sufficient. Mismatches come in equal numbers of `0 -> 1` and `1 -> 0`; one swap fixes one of each. Thus the minimum is half the Hamming distance.
- **Adjacent swaps → transport positions.** If source and target have equal one-count k, list their one-positions as $p_1 < ... < p_k$ and $q_1 < ... < q_k$. The minimum adjacent swaps is $\sum_i |p_i - q_i|$. Match in order; crossing identical ones adds no useful movement.
- **Sort by inversions.** To sort a binary string into all zeros then ones using adjacent swaps, count `10` subsequences. Each effective adjacent swap removes exactly one such inversion. A left-to-right scan adds the number of preceding ones whenever it sees zero.
- **Group all ones with adjacent swaps.** For one-positions p_i, choose a consecutive block starting at a. Its cost is $\sum_i |(p_i - i) - (a-1)|$, minimized by a median of $p_i - i$. Subtracting the index accounts for the space occupied by the other ones. With no ones, cost is zero.
- **Group ones with arbitrary swaps.** If there are k ones, choose a length-k window containing the most ones. Its number of zeros is the minimum swaps: each zero swaps with an outside one. Use a sliding window; circular grouping requires circular windows.
- **Alternating target under replacements.** There are only two alternating strings of a positive fixed length. Compute Hamming distance to both and take the smaller. Their mismatch counts sum to n, so the answer is at most $\lfloor \frac{n}{2} \rfloor$.
- **Alternating target under swaps.** First require $|z - o| \le 1$; for odd length the majority bit must start. For every feasible target, arbitrary swaps cost half its Hamming distance;

adjacent swaps cost the ordered one-position transport sum. Different operation models give different answers.

- **Lexicographic improvement with adjacent swaps.** To minimize with at most k swaps, process zeros in source order. Move each left across as many preceding ones as the remaining budget allows. Paying earlier has priority because the first differing bit decides lexicographic order.

4.4 Cancellation, masks, and interval observations

- **Delete either adjacent opposite pair.** Under `01 -> empty` and `10 -> empty`, the difference $z - o$ is invariant. Every nonconstant string has an opposite boundary, so a terminal string is constant. Its length is exactly $|z - o|$; exactly $\min(z, o)$ deletions occur, regardless of choices.
- **Delete adjacent equal pairs.** Under `00 -> empty` and `11 -> empty`, a stack yields a unique alternating remainder. Total length parity and each bit-count parity are invariant, but do not determine the answer: `0011` disappears while `0101` does not. This is different from both run compression and opposite-pair deletion.
- **One interval flip → signed gain.** To maximize ones after flipping one interval, give zero weight $+1$ and one weight -1. Interval gain is its sum. Maximize a prefix sum minus the smallest earlier prefix sum. If zero operations are allowed, also consider gain zero; an exactly-one requirement needs a nonempty interval.
- **At most k bit repairs → window.** The longest substring that can become all ones with at most k flips is the longest window containing at most k zeros. Two pointers work because enlarging a window cannot decrease its zero-count. For a constant substring, solve for both target bits.
- **Exactly $k > 0$ ones in a substring.** Let one-positions be $p_1, ..., p_o$, with sentinels $p_0 = 0, p_{o+1} = n + 1$. For each consecutive group $p_i, ..., p_{i+k-1}$, its contribution is $(p_i - p_{i-1})(p_{i+k} - p_{i+k-1})$. For $k = 0$, sum zero-run triangular counts instead.
- **Parity masks → palindrome-permutable ranges.** For any small alphabet, prefix XOR a one-hot bit per character. A range's parity mask is the XOR of its two prefix masks. It admits a palindrome permutation iff the mask has at most one set bit. This tests rearrangeability, not whether the range is already a palindrome.
- **Count ranges with few odd counts.** When scanning prefix parity masks M, previous equal masks give ranges with every count even; masks differing in one bit give exactly one odd count. Add their stored frequencies before inserting the current prefix. For binary strings, masks have only four states.
- **Antipalindromes are mirrored inequalities.** A binary antipalindrome satisfies $s_i \neq s_{n+1-i}$ for every position; its length must be even. With replacements, minimum cost is the number of equal mirrored pairs. With free rearrangement, feasibility is exactly equal zero/one counts.

5 Prefix transforms and range sums

Use one-based array indices, inclusive intervals $[l, r]$, and $P_0 = 0$, $P_i = \sum_{j=1}^{i} a_j$. A subarray corresponds to an ordered pair of prefix boundaries $i < j$; its length is $j - i$ and its sum is $P_j - P_i$.

5.1 Turn a subarray into two states

- **The empty prefix is a real boundary.** Seed frequency maps with `freq[0] = 1` and earliest-position maps with `first[0] = 0`. Otherwise every valid subarray beginning at index one disappears. There are $n + 1$ prefix boundaries, although only n elements.
- **Exact sum becomes a lookup.** For each right boundary j, a sum-k subarray needs an earlier prefix $P_i = P_j - k$. Add its frequency, then insert P_j. Inserting first incorrectly counts an empty subarray when $k = 0$; negative elements cause no difficulty.
- **Store the statistic the objective needs.** Frequencies count solutions; the earliest matching boundary gives the longest subarray; the latest gives the shortest nonempty subarray. A Boolean presence flag is sufficient only for existence. Do not overwrite earliest positions when maximizing length.
- **Equal prefix classes count zero sums.** If a prefix value appears f times, it supplies $\frac{f(f-1)}{2}$ zero-sum subarrays. The same grouping works for any symmetric equality condition. For a nonzero exact target, sorting prefixes without preserving index order generally loses the direction of subtraction.
- **Divisibility discards almost everything.** Sum divisible by positive m means equal prefix remainders modulo m. In C++, take `r = x % m` and add m only if $r < 0$. Count equal classes, including the empty prefix; large prefix values are irrelevant after reduction. Practice: Subarray Divisibility.
- **Nonzero residues retain direction.** Sum congruent to t requires an earlier remainder congruent to $P_j - t$, rather than simply equal remainders. Likewise, sum and length congruences combine into a tuple of prefix sum remainder and boundary-index remainder.
- **Balanced categories become vector equality.** For c symbols, keep prefix counts relative to one reference symbol: `(cnt[1]-cnt[c], ..., cnt[c-1]-cnt[c])`. Two equal vectors enclose equal counts of every category. For two symbols, mapping them to $+1, -1$ is the one-dimensional version.
- **Required ratios become linear invariants.** To require counts $A : B = p : q$ with $p, q > 0$, assign category A weight q and category B weight $-p$. A zero-sum interval satisfies the ratio equation. If other categories are allowed, exclude intervals with $A = B = 0$ when positive counts are required.
- **Average conditions are sum conditions.** Average at least x means the sum of $a_i - x$ is nonnegative. For rational $x = \frac{u}{v}$, use integer weights $va_i - u$ with $v > 0$ to avoid floating-point ambiguity. An exact average becomes equal transformed prefixes.
- **Inequalities become ordered prefix queries.** Sum at least k means $P_i \leq P_j - k$; sum in $[L, R]$ means P_i belongs to $[P_j - R, P_j - L]$. Coordinate compression plus an order-statistic count handles negative elements, where ordinary sum-based two pointers fail.
- **Maximum sum means minimum earlier prefix.** For a fixed right boundary, maximize $P_j - P_i$ by minimizing the allowed P_i. Maintain a running minimum for unrestricted nonempty subarrays. Query before adding the current boundary so an all-negative array cannot silently choose an empty interval.
- **Length bounds select a moving prefix window.** If allowed lengths are $[L, R]$, the candidate boundaries for right endpoint j are exactly $[j - R, j - L]$ intersected with $[0, n]$. Maintain frequencies for counting or a minimum for maximizing sum, inserting and expiring boundaries as this window moves. Practice: Maximum Subarray Sum II.
- **Prefix subtraction needs an inverse.** Sums and XOR support cancellation; minimum, maximum, OR, and gcd do not. Two prefix minima cannot reconstruct a range minimum. Modular products permit division only by invertible factors; zeros and nonunits require additional information.

5.2 Counting windows without enumerating them

- **Monotonicity is the real two-pointer condition.** With nonnegative elements, extending a window cannot decrease its sum, and removing its left element cannot increase it. Negative elements break both implications. Zero elements preserve monotonicity but introduce duplicate prefix values and multiple starts with the same sum.
- **Count all valid suffixes at once.** For a condition preserved by deleting endpoints, such as at most k distinct values, let l be the smallest valid start for endpoint r. Then exactly $r - l + 1$ nonempty windows end at r. This requires proving every later start remains valid.
- **Exactly is a difference of cumulative counts.** For an integer-valued statistic, `exactly(k) = atMost(k) - atMost(k-1)`. Examples include distinct values, odd elements, and binary ones. The identity always holds; an efficient two-pointer implementation additionally needs the appropriate monotonicity. Treat negative bounds as giving zero for nonnegative statistics.
- **Zeroes make exact-sum windows multiply.** In `[0,0,1]`, three subarrays ending at the last position sum to one. A routine retaining just one matching left endpoint undercounts. Prefix frequencies or the difference of two at-most counts automatically include all such choices.
- **Exactly k marked positions factors into gaps.** Let marked positions be $p_1 < ... < p_m$, with $p_0 = 0$, $p_{m+1} = n + 1$. For $k \geq 1$, a block of marks p_t through p_{t+k-1} contributes $(p_t - p_{t-1})(p_{t+k} - p_{t+k-1})$ intervals. Sum over possible first marks.
- **No marked positions means triangular gaps.** An unmarked run of length g contributes $\frac{g(g+1)}{2}$ empty-of-marks subarrays. Apply this to zero ones, no forbidden values, or no element exceeding a threshold. Sentinels handle the leading and trailing runs uniformly.
- **All elements pass a threshold iff the whole interval stays in one good run.** To count subarrays with minimum at least x, mark positions satisfying $a_i \geq x$ and sum triangular run lengths. For maximum at most x, instead use positions satisfying $a_i \leq x$.
- **An exact extremum is a threshold difference.** For integer values, `count(max = x) = count(max <= x) - count(max <= x-1)`. Similarly use `count(min >= x) - count(min >= x+1)`. This avoids assigning a special representative occurrence of the extremum.
- **Fixed bounds have a last-occurrence formula.** For subarrays whose minimum is L and maximum is R, with $L \leq R$, track the last invalid position b, last occurrence u of L, and last occurrence v of R. Endpoint r contributes $\max(0, \min(u, v) - b)$.
- **Parity and XOR use prefix states too.** A subarray XOR is the XOR of its two boundary prefixes. Exact XOR k therefore needs earlier state `prefixXor ^ k`. Equal states mean XOR zero; for a binary parity condition there are only two states.

5.3 Range operations become sparse boundary events

- **A range addition touches two differences.** With $D_i = a_i - a_{i-1}$ and $a_0 = 0$, adding x on $[l, r]$ adds x to D_l and subtracts x from D_{r+1}. Allocate a sentinel beyond n; one prefix scan reconstructs all final values.
- **A point change is also a boundary pair.** Changing a_i by x changes only D_i by x and D_{i+1} by $-x$. Conversely, changing one difference adds a suffix. Switching between values and differences can turn a global-looking operation into a local one.
- **Flips have XOR differences.** A binary flip on $[l, r]$ toggles events at l and $r + 1$. Prefix XOR gives the flip parity at each position. Ten overlapping flips are equivalent to their parity, provided the operations are unconditional toggles rather than assignments or state-dependent actions.
- **Nested batches need two difference arrays.** If a query applies operations numbered $[x, y]$, first difference over operation indices to compute each operation's multiplicity. Then add its range update multiplied by that multiplicity to the array's difference events. Linearity collapses both layers. Practice: Greg and Array.
- **Arithmetic progression updates need four second-difference events.** Add $s + (i - l)d$ on $[l, r]$ and let $v = s + (r - l)d$. In a zero-initialized second-difference buffer, add $s, d - s, -v - d, v$ at $l, l + 1, r + 1, r + 2$, respectively. Coincident positions must accumulate; two prefix scans reconstruct the update, which is then added to the original array.
- **Polynomial degree tells you how many differences to take.** Each finite difference lowers polynomial degree by one. An update polynomial of fixed degree d becomes boundary events after $d + 1$ differences, followed by $d + 1$ integrations. Derive the boundary coefficients carefully; truncating the polynomial creates events at both ends.
- **Range additions plus prefix queries reveal two weighted sums.** Since $a_j = \sum_{i \leq j} D_i$, `prefixSum(t) = (t+1) * sum(D[1..t]) - sum(i*D[i], i<=t)`. Thus two Fenwick trees on D_i and iD_i support dynamic range addition and range-sum queries.
- **Assignment is not additive superposition.** The difference-array trick combines independent additions, XORs, and other commuting invertible updates. Range assignment overwrites earlier values, so merely summing endpoint events is wrong; time order or a representation tracking the most recent covering assignment is needed.
- **Half-open events make tie semantics explicit.** An integer interval $[l, r]$ becomes $[l, r + 1)$, with start $+1$ and end -1. Apply all events at coordinate x before interpreting coverage on the slab from x to the next event. Geometric endpoint inclusion requires its own stated convention.
- **Compressed coordinates preserve order, not distance.** If adjacent event coordinates are x_i, x_{i+1}, their slab has width $x_{i+1} - x_i$. Multiply coverage or qualifying status by this width when measuring length or counting integer positions; counting compressed indices alone gives the wrong measure.

5.4 Weighted prefixes, rectangles, and repeated summation

- **Weights relative to the left endpoint split into fixed prefixes.** Precompute $P_t = \sum_{i \leq t} a_i$ and $W_t = \sum_{i \leq t} i a_i$. Then the weighted sum `sum((i-l+1)*a[i], i=l..r)` is `W[r]-W[l-1]-(l-1)*(P[r]-P[l-1])`.
- **Polynomial weights expand into moments.** For weights $(i - l)^2$, expand to $i^2 - 2li + l^2$ and maintain prefixes of $i^2 a_i$, $i a_i$, and a_i. Any fixed-degree polynomial in position and query endpoints reduces to a fixed number of such moment queries.
- **A prefix of prefixes sums many ranges.** Let $Q_t = \sum_{j=1}^{t} P_j$. The total of all sums starting at l and ending between l and r is `Q[r]-Q[l-1]-(r-l+1)*P[l-1]`. Reverse the roles for intervals sharing a right endpoint.
- **Rectangle sums are four corner values.** A two-dimensional prefix table gives the sum on rows $[x_1, x_2]$ and columns $[y_1, y_2]$ as `P[x2,y2]-P[x1-1,y2]-`

`P[x2,y1-1]+P[x1-1,y1-1]`. The overlap removed twice must be added back once. Practice: Forest Queries.

- **Rectangle additions are four corner events.** Add $v, -v, -v, v$ at `(x1,y1)`, `(x2+1,y1)`, `(x1,y2+1)`, `(x2+1,y2+1)`. Prefix-sum along both axes to reconstruct values. In d dimensions, the same inclusion-exclusion principle uses 2^d corners.

5.5 Reverse the order of counting

- **One element belongs to many subarrays.** Position i has i choices of left endpoint and $n-i+1$ choices of right endpoint. Therefore the sum of all subarray sums is $\sum_{i=1}^{n} a_i i(n-i+1)$. This is a contribution count, so negative values are allowed.
- **Fixed-length participation has clipped endpoints.** Position i belongs to length-k windows starting between `max(1,i-k+1)` and `min(i,n-k+1)`. Its coefficient is the nonnegative size of that integer interval. This works at the array edges, where the naive coefficient k fails.
- **Weighting each subarray by its length still has a closed coefficient.** The sum of lengths of all subarrays containing i is $i(n-i+1)\frac{n+1}{2}$. Multiply by a_i and sum to obtain the total of `length * subarraySum`. Pairing endpoint choices explains the average length $\frac{n+1}{2}$.
- **Two positions contribute by their outer endpoints.** For $i < j$, exactly $i(n-j+1)$ subarrays contain both. Thus the total squared subarray sum has diagonal terms $a_i^2 i(n-i+1)$ and cross terms $2a_i a_j i(n-j+1)$. A running weighted prefix evaluates the cross terms linearly.
- **Distinct-value totals use the previous equal occurrence.** Let p_i be the previous position containing a_i, or zero. Position i is the first occurrence of its value in exactly $(i-p_i)(n-i+1)$ subarrays. Summing these coefficients gives the sum of distinct counts over all subarrays.
- **Runs are one plus the number of internal changes.** An unequal adjacent pair at positions $i, i+1$ lies inside $i(n-i)$ subarrays. Hence the total number of constant runs across all nonempty subarrays is $\frac{n(n+1)}{2}$ plus the sum of those coefficients over unequal adjacent pairs.
- **Point updates often change a huge aggregate locally.** For a linear aggregate $F = \sum_i w_i a_i$ with fixed weights, changing a_i by d changes F by dw_i. For the run-count aggregate above, only boundaries next to the changed position can change, despite affecting many subarrays.
- **All query sums reduce to coverage frequencies.** If c_i queries contain position i, the sum of their answers is $\sum_i c_i a_i$. Obtain c_i with boundary events. If values may be permuted freely, sorting values and coefficients in the same order maximizes this dot product.
- **Sorted pair distances become prefix sums.** For sorted values $b_1 \le ... \le b_n$, value b_j contributes $(j-1)b_j - \sum_{i<j} b_i$ to all earlier absolute differences. Equivalently, the total is $\sum_{i(2i-n-1)} b_i$. Sorting is essential to remove the absolute value correctly.
- **Pair XOR splits into independent bits.** For nonnegative fixed-width integers, bit b contributes $2^b c_{b(n-c_b)}$ to the sum over unordered pairs, where c_b values have that bit set. For the sum of all subarray XOR values, apply this to the $n+1$ prefix XOR states instead.
- **Minima need unique ownership when values tie.** Let L_i be the nearest position left of i with strictly smaller value, and R_i the nearest position right with smaller-or-equal value, using sentinels $0, n+1$. Position i owns $(i-L_i)(R_i-i)$ subarrays whose minimum is a_i.
- **One strict side and one nonstrict side prevent double counting.** Monotonic stacks find the boundaries above. Swap which side permits equality if desired, but do not make both sides strict or both nonstrict. Reverse comparisons for maxima; subtract total minima from total maxima for the sum of subarray ranges.

5.6 Boundaries, circularity, and transport

- **Circular intervals become ordinary intervals with restrictions.** Double the array, allow starting positions only in the first copy, and cap length at n. Without both restrictions, the same start may be counted twice or elements may repeat. Specify whether full-circle intervals with different starts are distinct in the problem.
- **Offline threshold queries share one evolving array.** Sort elements by value and queries by threshold, activating eligible positions before answering each query. For a `<= x` predicate, equal-valued elements must be active first; for `< x`, answer first. Ties are part of the proof, not an implementation detail.
- **Adjacent transfers become forced prefix flows.** To turn integer array a into target b by moving units between neighbors, total sums must agree. Let $F_i = \sum_{j=1}^{i(a_j-b_j)}$. Exactly F_i units of net flow must cross edge i to the right; a negative value means leftward flow.
- **Transport cost is the absolute area between prefix curves.** If either transfer direction is allowed and each unit crossing edge i costs $w_i \ge 0$, the minimum cost is $\sum_{i=1}^{n-1} w_i |F_i|$. This assumes unit transport with no capacity limits; nonnegative source and target inventories can be scheduled feasibly.
- **Direction restrictions turn cost formulas into feasibility tests.** If units may move only right, transformation is possible precisely when totals agree and every $F_i \ge 0$, for unrestricted integer inventories or nonnegative inventories with legal unit transfers. Every prefix must initially contain enough mass for its target; suffix conditions are the same statement viewed backward.

6 Number theory

Unless stated otherwise, moduli and divisors are positive, factorizations concern positive integers, and gcd uses absolute values. An invariant proves impossibility; proving sufficiency additionally requires a legal construction.

6.1 Gcd, differences, and reachable values

- **Euclid is an operation model.** Replacing (a, b) by $(a, b-ka)$ preserves gcd for every integer k. Repeated legal subtraction or remainder operations therefore suggest Euclid. Gcd preservation alone does not describe every reachable state when signs, bounds, or directions are restricted.
- **Global shifts disappear in differences.** For $n \ge 2$, let $D = \gcd(a_2 - a_1, ..., a_n - a_1)$. Then $\gcd(a_1 + x, ..., a_n + x) = \gcd(a_1 + x, D)$. Precompute D for many shift queries. If all entries coincide, $D = 0$ and the answer is $|a_1 + x|$.
- **Equal residues are a difference condition.** All a_i have the same residue modulo k exactly when k divides every $a_i - a_1$. Thus candidate positive k are divisors of D; if $D = 0$, every positive modulus works. Adjacent differences have the same gcd as differences from one anchor.
- **Independent fixed-step changes.** With unrestricted operations $a_i \leftarrow a_i \pm k$, a target vector is reachable exactly when each target entry has the same residue as its original entry modulo k. Equalization is possible exactly when all entries

share one residue. One-way increments add a target lower bound.

- **Several reversible step sizes.** On the unrestricted integer line, moves by $\pm d_1, ..., \pm d_t$ reach exactly the starting residue class modulo $\gcd(d_1, ..., d_t)$. This is Bézout's identity viewed as a construction; intermediate position bounds can invalidate it.
- **Nonnegative coefficients are a separate constraint.** For positive a, b, $ax + by = c$ with integer x, y is solvable iff $\gcd(a, b)$ divides c. Requiring $x, y \geq 0$ needs extra work: with steps $4, 6$, gcd divides 2 but 2 is not a nonnegative combination.
- **Circular moves split into gcd cycles.** Repeatedly adding k modulo n visits $\frac{n}{\gcd(n,k)}$ positions. Two indices communicate exactly when their difference is divisible by $\gcd(n, k)$. Use this for rotation or periodic-index operations; this describes positions, not arbitrary permutations within a cycle.
- **Adding multiples preserves a modular gcd.** Operations $a_i \leftarrow a_i + k_i m$ preserve $\gcd(a_1, ..., a_n, m)$ and each individual residue modulo m. The gcd is a compressed necessary condition; the full residue vector contains more information.
- **Normalize a common factor.** If every input and target is divisible by g, divide by g when all operations scale compatibly. Gcd or divisibility problems often become coprime after normalization; fixed additive constants and absolute costs must be scaled too.
- **All Diophantine solutions lie on one integer line.** For positive a, b, $g = \gcd(a, b)$, and one solution (x_0, y_0) to $ax + by = c$, every solution is $x = x_0 + t\frac{b}{g}$, $y = y_0 - t\frac{a}{g}$. Bounds become an interval of integers t; a linear objective is minimized at an endpoint.
- **Remainder equations turn into divisors.** For nonnegative a and desired remainder r, $a \bmod k = r$ means k divides $a - r$ and $0 \leq r < k$. If $a = r$, every $k > r$ works. Forgetting $k > r$ produces false divisor candidates.
- **A proper remainder shrinks fast.** If $a \geq k > 0$, then $a \bmod k < \frac{a}{2}$: either $k \leq \frac{a}{2}$, or the remainder is $a - k$. Thus each effective modulo update halves a nonnegative value. Charge changes to logarithmic shrinkage; updates with $k > a$ do nothing.

6.2 Congruences and modular arithmetic

- **Safe modular cancellation changes the modulus.** From $ax \equiv ay$ modulo m, conclude $x \equiv y$ modulo $\frac{m}{\gcd(a,m)}$, not necessarily modulo m. Example: $2x \equiv 2y$ modulo 6 only forces equality modulo 3.
- **Linear congruences may have several solutions.** For $ax \equiv b$ modulo m, let $g = \gcd(a, m)$. There is a solution iff g divides b; divide all three quantities by g, then solve using an inverse when $\frac{m}{g} > 1$. If $\frac{m}{g} = 1$, every integer satisfies the reduced congruence. There are exactly g solution classes modulo m.
- **Division requires an inverse.** An inverse of a modulo $m > 1$ exists iff $\gcd(a, m) = 1$. For prime p, a^{p-2} works when a is nonzero modulo p. An integer quotient must not be replaced by inverse multiplication when its denominator is noninvertible.
- **Exact integer cancellation can precede reduction.** To evaluate an integer expression such as $\frac{n(n-1)}{2}$ modulo an even modulus, divide the even integer factor by 2 before reducing. This avoids a nonexistent inverse; establish divisibility before cancelling.
- **Multiplication permutes residues only when coprime.** The map $x \to ax$ modulo m is bijective iff $\gcd(a, m) = 1$. Otherwise its image is the multiples of $g = \gcd(a, m)$ and every image residue has g preimages. Useful for counting collisions or testing whether information was lost.
- **Exponent reduction has hypotheses.** Euler permits reducing an exponent modulo $\varphi(m)$ only when the base is coprime to m. For prime p, use period $p - 1$ for nonzero bases. For noncoprime bases, track prime-power divisibility or use a theorem that explicitly handles the transient prefix.
- **CRT checks compatibility, then gives a period.** The constraints $x \equiv a$ modulo m and $x \equiv b$ modulo n are compatible iff $\gcd(m, n)$ divides $a - b$. If compatible, all solutions form one residue class modulo $\operatorname{lcm}(m, n)$. Merge constraints incrementally.
- **Repeated events need offsets as well as periods.** Events at $a + km$ and $b + jn$ meet by the preceding CRT condition. Their common events repeat every lcm, but the first nonnegative meeting need not equal the lcm. Enforce starting-time bounds after solving the congruences.
- **Residue sums give a guaranteed subarray.** Among the $n + 1$ prefix sums of any length-n integer array, two agree modulo n. Their difference is a nonempty subarray with sum divisible by n. Include the empty prefix; positivity is unnecessary.
- **Pigeonhole collisions are not always subsets.** Two subsets with equal sum modulo m give a relation with coefficients in $\{-1, 0, 1\}$ after cancelling their intersection. This does not automatically yield a nonempty subset summing to zero, because subtraction introduces negative coefficients.
- **Parity is the cheapest modulus.** A sum's parity equals the parity of its number of odd terms; a product is odd iff every factor is odd. If each move changes a statistic by an even number, its parity is fixed. Test several small moduli before searching for a complicated invariant.
- **Squares have few small-modulus residues.** Squares are $0, 1$ modulo 4 and $0, 1, 4$ modulo 8; odd squares are 1 modulo 8. Use these as impossibility filters, not sufficient existence conditions. More generally, enumerate all residues of a polynomial modulo a small modulus.

6.3 Prime exponents, divisors, and multiplicative operations

- **Products become exponent sums.** Write $v_{p(x)}$ for the exponent of prime p. Multiplication adds exponents, division subtracts them, gcd takes coordinatewise minima, and lcm takes maxima. Problems about redistributing factors often become independent accounting constraints for each prime.
- **The two-number gcd-lcm identity.** For positive a, b, $ab = \gcd(a, b)\ \operatorname{lcm}(a, b)$. Compute lcm as $\left(\frac{a}{\gcd(a,b)}\right)b$ to reduce intermediate overflow. For three or more numbers, the product generally differs from gcd times lcm.
- **Prescribed gcd and lcm remove a scale factor.** If $\gcd(a, b) = g$ and $\operatorname{lcm}(a, b) = L$, require g to divide L. Write $a = gx, b = gy$: then $\gcd(x, y) = 1$ and $xy = \frac{L}{g}$. Each complete prime-power factor of $\frac{L}{g}$ must go wholly to one side.
- **Perfect powers are exponent congruences.** A positive integer is a perfect kth power iff all prime exponents are divisible by k. Store the exponent vector modulo k to forget irrelevant factors. For $k = 2$, the product of primes with odd exponents is the square-free kernel.
- **Pair products use complementary signatures.** For positive a, b, ab is a square iff their square-free kernels agree. For a kth power, match exponent residue r with $(-r) \bmod k$ for every prime. Handle zero separately and account for sign when signed values are allowed.
- **Dividing by allowed prime factors leaves a core.** If multiplication and exact division by primes in a fixed set

are reversible and unrestricted, two positive numbers communicate iff removing all those primes leaves the same core. With division-only operations, their possible common descendants are constrained further.

- **Factor transfers conserve each prime's total.** If an operation can transfer one factor p from any divisible entry to any other entry, a positive length-n array can become constant iff every prime's total exponent is divisible by n. This equivalence relies on unrestricted transfers and sufficient divisibility at each donor.
- **Count divisors by exponent choices.** If $x = \prod p_i^{e_i}$, its positive-divisor count is $\prod(e_i + 1)$ and divisor sum is $\prod\left(1 + p_i + ... + p_i^{e_i}\right)$. The divisor count is odd exactly for perfect squares. Modular geometric-series division needs the usual inverse check.
- **Pair divisors around the square root.** Every divisor d has partner $\frac{x}{d}$, so enumerate $d \le \sqrt{x}$ and add both when distinct. Use `d <= x / d` to avoid overflow in `d*d`; do not double-count the square root of a perfect square.
- **Factorials count repeated prime contributions.** $v_{p(n!)} = \sum_{j \ge 1} \left\lfloor \frac{n}{p^j} \right\rfloor$. Implement by repeatedly dividing a running n by p to avoid overflow in p^j. For $K = \prod p_i^{e_i} > 1$, the largest t with K^t dividing $n!$ is $\min_i \left\lfloor v_{p_i} \frac{n!}{e_i} \right\rfloor$.
- **Factorial ratios can be checked without constructing them.** The exponent of p in $\frac{n!}{r!(n-r)!}$ is the difference of three factorial valuations. A prime dividing the numerator factorial need not divide the final binomial coefficient: denominator cancellation matters.
- **Trial division has a precise stopping rule.** After removing factors through the current square root, any remaining value greater than 1 is prime. A composite must have a prime factor at most its square root. Recompute the bound against the shrinking remainder for faster factorization.
- **Few distinct primes fit into a bounded number.** If x has t distinct prime factors, their product divides x, so at least the product of the first t primes must fit. This makes subset inclusion-exclusion over distinct prime factors practical for many moderate input bounds.
- **Count coprime values by prime-factor inclusion-exclusion.** To count $1 \le y \le N$ with $\gcd(y, x) = 1$, exclude multiples of the distinct primes dividing x. A selected prime subset contributes $\left\lfloor \frac{N}{\prod p} \right\rfloor$ with alternating sign. Exponents of those primes do not affect coprimality.
- **Consecutive integers are coprime.** $\gcd(x, x+1) = 1$; more generally $\gcd(x, x+k) = \gcd(x, k)$. In a product of consecutive integers, distribute divisibility obligations among the terms. The product of k consecutive positive integers is divisible by $k!$.

6.4 Floors, multiples, and harmonic counting

- **Count multiples by endpoint division.** For integers $L \le R$ and $d > 0$, the number of multiples of d in $[L, R]$ is $\left\lfloor \frac{R}{d} \right\rfloor - \left\lfloor \frac{L-1}{d} \right\rfloor$. This includes negative intervals when division means mathematical floor; ordinary C++ integer division truncates toward zero.
- **Ceil without overflow.** For $a \ge 0, b > 0$, $\left\lceil \frac{a}{b} \right\rceil$ equals `a / b + (a % b != 0)`. The usual `(a+b-1)/b` can overflow. For signed numerators, use sign-aware floor/ceil division; $\left\lceil \frac{a}{b} \right\rceil = -\left\lfloor -\frac{a}{b} \right\rfloor$ also needs care when negating the minimum integer.
- **First feasible multiple or congruent value.** The least multiple of $d > 0$ at least L is $d\left\lceil \frac{L}{d} \right\rceil$. More generally the least $x \ge L$ with $x \equiv r$ modulo d is $r + d\left\lceil \frac{L-r}{d} \right\rceil$. Then check the upper bound.
- **Normalize remainders deliberately.** Mathematical residues lie in $[0, m)$. In C++, compute `r = x % m; if (r < 0) r += m;` for positive m. This avoids the possible overflow of `(x % m + m) % m` when the first remainder is already nonnegative.
- **Rounding losses expose carries.** For nonnegative integers a_i and $d > 0$, $\left\lfloor \frac{\sum a_i}{d} \right\rfloor - \sum \left\lfloor \frac{a_i}{d} \right\rfloor = \left\lfloor \frac{\sum (a_i \bmod d)}{d} \right\rfloor$. For two inputs the loss is 0 or 1; for n inputs it is at most $n - 1$.
- **Many quotients are identical.** For fixed $N > 0$ and current $l \le N$, let $q = \left\lfloor \frac{N}{l} \right\rfloor$. This quotient stays constant through $r = \left\lfloor \frac{N}{q} \right\rfloor$. Process $[l, r]$ together and jump to $r + 1$; there are only $O\left(\sqrt{N}\right)$ groups.
- **Sum by divisors instead of by values.** $\sum_{i=1}^{N} \left\lfloor \frac{N}{i} \right\rfloor$ counts pairs (i, j) with $ij \le N$, equivalently the total number of positive divisors of $1, ..., N$. Switch the order of counting whenever an inner loop asks whether one number divides another.
- **Multiples loops have harmonic total work.** Visiting $d, 2d, ...$ for every $1 \le d \le V$ costs $\sum_d \left\lfloor \frac{V}{d} \right\rfloor = O(V \log V)$. This supports divisibility-frequency tables over a bounded value domain even when iterating over every pair of input elements is too slow.
- **Recover exact gcd counts from divisibility counts.** For positive values at most V, let c_d count entries divisible by d. Then $\binom{c_d}{2}$ counts unordered distinct-index pairs whose gcd is a multiple of d. Descending over d, subtract counts already assigned to proper multiples to obtain exact-gcd counts.
- **Gcd chains have few distinct positive values.** Extending a range can only replace its gcd by a divisor. A strict decrease of a positive gcd at least halves it, so suffix gcds at one endpoint have $O(\log V)$ distinct values. Merge equal states; zeros may add an initial gcd-0 state.
- **Linear floor sums admit Euclidean reduction.** For $\sum_{i=0}^{n-1} \left\lfloor \frac{ai+b}{m} \right\rfloor$, first split off the integer quotients of $\frac{a}{m}$ and $\frac{b}{m}$. The remaining lattice-point count can swap its axes and recurse like Euclid. This is useful when quotient grouping alone still leaves too many terms.

6.5 Digits as modular weights

- **Digit sum is a modular invariant.** In base B, $B \equiv 1$ modulo $B - 1$, so a number and its digit sum agree modulo $B - 1$. Permuting digits preserves this residue. In decimal, divisibility by 3 and 9 follows immediately; residue alone does not determine the number.
- **Alternating digit sums come from minus one.** Since $B \equiv -1$ modulo $B + 1$, use the alternating sum of digits from the units place. In decimal this tests divisibility by 11; arbitrary digit permutations do not preserve the alternating sum.
- **Concatenation is an affine operation.** Appending a base-B block y of length k maps x to $B^k x + y$. Work modulo the desired modulus, retain block lengths including leading zeros, and precompute powers. Repeating blocks forms a geometric sum; division by $B^k - 1$ may be invalid modulo the target.
- **Suffix tests depend on the modulus dividing a base power.** The last k digits determine the value modulo m iff m divides B^k. For decimal, such moduli have only prime factors 2 and 5; split a general modulus into this part and its coprime part when helpful.

7 Bitwise observations

Use nonnegative integers or a fixed-width unsigned representation. Bits are independent for XOR/AND/OR, but addition couples adjacent bits through carries. In formulas below, XOR, AND, and OR are written as code operators `^`, `&`, and `|`.

7.1 Carries, parity, and bit contributions

- **Addition separates into XOR and carries.** `a + b = (a ^ b) + 2 * (a & b)` as an integer identity. Hence `(a+b) == (a^b)` iff `(a&b) == 0`. Use a wider type for the arithmetic; fixed-width overflow can hide the distinction.
- **Sum and OR have a second decomposition.** `a + b = (a | b) + (a & b)`. Consequently XOR is at most OR, which is at most the ordinary sum, for nonnegative operands. Equality conditions often turn an arithmetic target into disjoint-bit requirements.
- **For many summands, no carries means disjoint set bits.** A nonnegative sum equals the XOR of all summands iff no bit is set in more than one summand. Count set occurrences per bit: ordinary addition uses the full count, whereas XOR retains only its parity.
- **Construct a pair from its sum and XOR.** To solve $a + b = S$ and `a ^ b = X` for nonnegative a, b, require $S \geq X$, even $S - X$, and `(((S-X)/2) & X) == 0`. Let $C = \frac{S-X}{2}$; then `a=C`, `b=C|X` is one solution. Strict positivity adds constraints.
- **XOR parity is per-bit frequency parity.** Duplicates occurring an even number of times cancel. XOR of an array identifies the odd-frequency value only when the problem guarantees exactly one such value. Several odd-frequency values are compressed together and cannot generally be recovered from one XOR.
- **Pairwise XOR sums separate by bit.** Among n values, if c_b have bit b set, unordered distinct-index pairs contribute $c_{b(n-c_b)}2^b$. For AND use $\binom{c_b}{2}2^b$; for OR use $\left(\binom{n}{2} - \binom{n-c_b}{2}\right)2^b$.
- **A common XOR mask has an independent sum effect.** XORing every element by 2^b changes the total sum by $(n - 2c_b)2^b$. If each mask bit is freely selectable, optimize each independently. Bounds or a required popcount on the mask couple the choices again.
- **Numbers with disjoint bits can still be large.** If `(a & b) == 0`, then adding one to an operand may create carries and destroy the property. Bitwise constraints are not monotone in numeric order; do not assume ordinary binary search is valid without a separate monotone predicate.

7.2 Reversible XOR operations and invariants

- **A common XOR mask preserves pairwise XOR.** `(a ^ x) ^ (b ^ x) = a ^ b`. To map one indexed array to another by one global mask, the values `a[i] ^ target[i]` must all coincide. For multiset matching, try masks from one fixed source value to candidate target values, then verify frequencies.
- **Whole-array XOR depends on length parity.** Applying the same mask x to all n entries changes their aggregate XOR by x when n is odd and leaves it unchanged when n is even. Pairwise XOR remains unchanged in both cases.
- **A paired toggle preserves total XOR.** Applying the same mask to two distinct positions cancels from the array XOR. If arbitrary masks can be applied to any pair, equality of total XOR is also sufficient for reachability: fix positions one by one using the last position as a buffer.
- **Connected-edge toggles are component constraints.** If a move XORs both endpoints of an edge by an arbitrary mask, each connected component's aggregate XOR is invariant and sufficient for reachability. Route corrections along a spanning tree. A single fixed permitted mask gives extra per-vertex restrictions.
- **Row-style XOR updates preserve span, not total XOR.** With distinct indices, replacing `a[i]` by `a[i] ^ a[j]` is its own inverse and preserves the XOR linear span and its rank. The aggregate XOR generally changes. This is the right model for operations resembling elimination over binary vectors.
- **Subset XORs form a vector space.** If an XOR basis has rank r, exactly 2^r XOR values can be made from subsets, including the empty subset. Membership can be tested by elimination. This assumes any subset is allowed; contiguity, cardinality, or other selection restrictions need extra information.
- **Fixed-mask toggles care only about parity.** Applying the same XOR mask twice is the identity, and XOR updates commute. A sequence therefore reduces to whether each available mask is used an odd or even number of times. Exact operation counts can still matter if redundant pairs cannot be inserted legally.
- **AND and OR are idempotent, not reversible.** Repeating `x &= mask` or `x |= mask` changes nothing after its first application. A chain of AND updates combines its masks by AND; a chain of OR updates combines by OR. Mixing the two requires respecting order.

7.3 Monotone bit sets and small state spaces

- **Range AND and OR change only a few times.** Extending a range can only clear bits of its AND or set bits of its OR. With W relevant bits, there are at most $W + 1$ distinct values along one extension chain. Merge equal suffix states for counting or finding ranges.
- **AND and OR define necessary endpoint bounds.** Every range AND is a submask of each member, and every range OR is a supermask of each member. For nonnegative values this implies AND is at most the minimum and OR at least the maximum. Numerical inequality alone does not imply the submask relation.
- **AND of an integer interval is its common prefix.** For all integers in $[L, R]$, retain their common high-bit prefix and zero every bit at and below the highest differing bit. The range OR keeps that prefix and sets every bit at and below that position. Special-case $L = R$; this concerns consecutive integers, not arbitrary array ranges.
- **The highest differing bit decides order.** For nonnegative $a \neq b$, the most significant set bit of `a ^ b` determines which number is larger. Under a common XOR mask, only that mask bit decides whether their order reverses; lower bits cannot overcome it.
- **Submasks are subsets of set bits.** `(x & mask) == x` means x is a submask. Enumerate nonzero submasks with `s = (s - 1) & mask`; process zero separately and stop there. A mask with k set bits has 2^k submasks, regardless of its numerical magnitude.
- **All mask-submask pairs cost three choices per bit.** Across every W-bit mask and every submask, total pairs are 3^W: each bit is absent, in the mask only, or also in the submask. This bounds a common nested enumeration even when no dynamic programming is involved.

7.4 Useful binary arithmetic identities

- **Erase or isolate the lowest set bit.** For positive unsigned x, `x & (x-1)` removes its lowest set bit and `x & -x` isolates it using unsigned wraparound. Repeated erasure takes popcount-many iterations. The test for a power of two must also require $x \neq 0$.
- **Low bits encode powers-of-two remainders.** For nonnegative x, keeping its lowest k bits gives $x \bmod 2^k$: use `x & ((1ULL << k)-1)` when k is less than the type's width. Right shift by k gives $\left\lfloor \frac{x}{2^k} \right\rfloor$. Trailing zero count gives $v_2(x)$ for $x > 0$; zero needs separate handling in many compiler intrinsics.

- **XOR of an integer prefix repeats every four.** The XOR of $0, 1, ..., n$ is respectively $n, 1, n+1, 0$ when $n \bmod 4$ is $0, 1, 2, 3$. The XOR of $[L, R]$ follows by cancelling two prefixes; define the empty prefix separately when $L = 0$.
- **One bit follows a periodic pattern.** Bit b is zero for 2^b consecutive integers, then one for 2^b, repeating. To count its set occurrences in $[0, N]$, count complete periods of length 2^{b+1} and the one-block part of the remainder. Widen $N + 1$ and period computations when needed.
- **Subtracting from an all-ones mask is complementation.** For $0 \le x < 2^W$, $(2^W - 1) - x$ equals XOR with the W-bit all-ones mask. Thus their sum is fixed and their AND is zero. The width restriction matters: an unrestricted bitwise complement includes additional high bits.
- **Bit width is part of the problem.** Use an unsigned literal of sufficient width for shifts; shift counts must be nonnegative and smaller than that width. Signed overflow, shifting into an unrepresentable result, and computing 2^W in a W-bit type can invalidate otherwise correct identities.

8 Worked observation chains

8.1 Neighbor transfers become prefix flow

Turn $[3, 1, 2]$ into $[1, 2, 3]$ by moving one unit between neighbors. Their totals agree. The prefix imbalances are $F_1 = 2, F_2 = 1$: two units must cross the first edge rightward and one must cross the second. Every move crosses one edge, so at least three moves are needed. Send two units from position 1 to 2, then one from 2 to 3; the bound is attained. The reduction gives both the cost and a construction.

8.2 Adjacent simultaneous increments expose an alternating sum

For $[1, 1, 1] \to [2, 3, 2]$, the required increase is $[1, 2, 1]$. Let x_1, x_2 count increments of pairs $(1, 2)$ and $(2, 3)$. The equations force $x_1 = 1, x_1 + x_2 = 2, x_2 = 1$, so the target is reachable in two moves. The alternating sum remains 1. Target $[2, 2, 2]$ has alternating sum 2, so it is impossible under this operation.

8.3 Binary flips become run boundaries

For source `001101` and target `011001`, the mismatch string is `010100`. It has two one-runs, so two arbitrary interval flips suffice: flip positions 2 and 4 separately. Extending mismatches by zero at both ends gives four transition boundaries. A single interval flip toggles only two boundaries, proving that one operation cannot suffice. This argument also works for overlapping intervals.

8.4 A global shift disappears from differences

For array $[6, 14, 22]$, the anchor differences are 8 and 16, with gcd 8. After adding an arbitrary integer x to every element, the new gcd is $\gcd(6 + x, 8)$. For $x = 2$, the array becomes $[8, 16, 24]$ and its gcd is 8. A query on the entire array has become one gcd computation involving a fixed precomputed number.

8.5 Signed subarray sums become matching boundaries

For $[2, -3, 2]$, the prefix sums are $[0, 2, -1, 1]$. To count sum -1 subarrays ending at boundary j, look for earlier $P_i = P_j + 1$. At $j = 2$ this matches the empty prefix; at $j = 3$ it matches prefix 2. The two intervals are $[1, 2]$ and $[2, 3]$. Querying before insertion enforces nonempty intervals, and the negative element causes no special case.

8.6 A noninvertible factor changes the modulus

Suppose a transformation requires $6x \equiv 8$ modulo 14. Since $\gcd(6, 14) = 2$ divides 8, divide by 2 and solve $3x \equiv 4$ modulo 7. The inverse of 3 modulo 7 is 5, so $x \equiv 6$ modulo 7. There are two solutions modulo 14: 6 and 13. Dividing the original congruence while retaining modulus 14 would discard valid solutions.

9 Final checks before coding

- **Name the operation precisely.** Swaps versus replacements, arbitrary positions versus adjacent positions, exactly versus at most, signed versus nonnegative intermediate values, and fixed versus freely chosen masks produce different problems.
- **Check both directions of an iff.** Prove every legal move preserves the proposed condition; then show how to reach every target satisfying it. If only the first direction is known, label the condition necessary.
- **Separate feasibility, construction, and minimum cost.** A gcd or parity test may settle feasibility while leaving the operation count unresolved. A construction gives an upper bound; match it with a lower bound before claiming optimality.
- **Audit endpoints and ties.** Empty prefixes, length one, all equal, all zero, odd length, duplicate values, closed interval ends, and equal extrema are where elegant formulas most often need correction.
- **Keep the arithmetic model honest.** Check negative remainder normalization, signed floor division, modular inverses, multiplication overflow, zero in gcd/factorization, and bit width. Reducing an answer modulo a number can destroy order comparisons.
- **Match the complexity to constraints.** A value bound permits frequency arrays and multiples loops; an alphabet bound permits masks; a bit-width bound permits distinct AND/OR states; a huge repetition count suggests cycles or composition.
- **Record the reason beside the trick.** "Prefix sum" is a tool name; "equal boundary states enclose a balanced interval" explains why to use it and what to store. A short proof makes a trick reusable when the story changes.

10 References for deeper proofs

The notes above give operation assumptions and derivations; these author-maintained references expand the longer algorithms. Relevant official practice problems are linked in the range section.

- Linear Diophantine equations: Bézout, parameterizing solutions, and bounded coefficients.
- Chinese remainder theorem and modular inverses: compatibility and valid modular division.
- Factorial prime valuations: counting prime factors without constructing a factorial.
- AtCoder Library floor-sum implementation: Euclidean reduction for linear floor sums.
- Prefix function, Lyndon factorization, and string hashing: borders, minimal rotations, and probabilistic equality checks.
- Fine–Wilf periodicity proof, Lemma 1: why sufficient overlap forces the gcd period.